

Gödel's Second Incompleteness Theorem for Binary Recursive Arithmetic in Agda

Thierry Coquand

May 2026

Abstract

We formalize Gödel's second incompleteness theorem (Gödel II) for *Binary Recursive Arithmetic* (BRA), a Rose/Guard-style equational system on binary trees with explicit substitution and primitive recursion. The development is in Agda with `--safe --without-K --exact-split`, ASCII-only, no postulates, no holes. The headline result is `thmT` is sound, and `Deriv Con` $\rightarrow$ `Deriv  $\perp$` . The proof is constructive and consists of an internalised proof checker `thmT` (Theorem 11), a Theorem 12 closure expressing that all primitive-recursive functions of BRA are reflected in `thmT`, a Theorem 14 closure linking `thmT` to BRA's encoding of provability, and a final `Compose` module producing `Deriv Con` $\rightarrow$ `Deriv  $\perp$` .

1 Overview

The BRA codebase lives in `BRA/`; this document summarises its main result. The chain is

`Thm 11 (concrete)` $\rightarrow$ `Thm 12 (parametric)` $\rightarrow$ `Thm 13` $\rightarrow$ `Thm 14` $\rightarrow$ `Compose` $\rightarrow$ `godelII`

assembled in `BRA/GoedelIIIFull.agda`, whose top-level statement is

```
godelIII : Deriv Con -> Deriv bot
```

After the soundification work described in Section ??, the meaning of `Con` is robust: every output of `thmT` on a syntactically well-formed proof tree is either a valid encoded formula `codeFormula(P)` or the explicit safe default `codeTriv = falseT = codeFormula(0 = 0)`, so `Con` $\equiv \neg \text{Pr}(\text{codeFormula}(\perp))$ truly expresses BRA's consistency.

2 The base system

2.1 Trees, terms, formulas

Definition 1 (Tree). `Tree` ::= `lf` | `nd Tree Tree`.

Definition 2 (Term, Functors). `Term` has constructors `O`, `var n` for $n : \mathbb{N}$, `ap1 f t` for $f : \text{Fun}_1$, and `ap2 g t1 t2` for $g : \text{Fun}_2$. `Fun1` and `Fun2` are inductive types: `Fst`, `Snd`, `Lift g`, `Comp f g`, `KT t` $\in \text{Fun}_1$; `Pair`, `lfLf`, `Fan f g h`, `Post f g` $\in \text{Fun}_2$, etc.

Definition 3 (Formula). `Formula` ::= `atomic (eqn tl tr)` | `not P` | `P imp Q`.

2.2 Derivability

$\text{Deriv} : \text{Formula} \rightarrow \text{Set}$ is an inductive predicate with explicit constructors for the BRA axioms (axI , axFst , axSnd , axComp , axComp2 , axLift , axPost , axFan , axKT , axIfLfL , axIfLfN , $\text{axTreeEq}[\text{LL}/\text{LN}/\text{NL}/\text{NN}]$, axGoodstein , axRefl , axK , axS , axNeg , axContrapos , axExFalso , axEqTrans , $\text{axEqCong}[\text{L}/\text{R}]$, axRecLf , axRecNd , axRecPLf , axRecPNd , axFstLf , axSndLf , axIfLfLL , axIfLfNL , axConst) and the inductive rules ruleSym , ruleTrans , cong_1 , cong_L , cong_R , mp , ruleInst , ruleInst_2 , ruleIndBT .

2.3 Encoding and reify

Each Term has a tree code $\text{code} : \text{Term} \rightarrow \text{Tree}$; each Formula has a code $\text{codeFormula} : \text{Formula} \rightarrow \text{Tree}$. Trees lift to terms via $\text{reify} : \text{Tree} \rightarrow \text{Term}$. The compositions $\text{reify} \circ \text{code}$ and $\text{code} \circ \text{reify}$ are not the identity in general, but $\text{reify} \circ \text{codeFormula } P$ is always Pair-shaped because $\text{codeFormula}(P) = \text{nd } _ _$ for every P .

3 Theorem 11: the proof checker thmT

Definition 4 (thmT). $\text{thmT} : \text{Fun}_1$ is a one-argument BRA function, defined inside the abstract block of `BRA/Thm/ThmT.agda` via a *treeRec*-style fold over its proof-tree input. At each tag tagCode_X , thmT dispatches to a body $\text{body}_X : \text{Fun}_2$ that computes the equation code from the proof's payload and sub-proof images.

Theorem 5 (Thm 11, the dispatchers). For every BRA derivation rule X with encoder enc_X and result out_X , there is a closed-form lemma thmTDispX in `BRA/Thm/ThmT.agda` of the shape

$$\forall (IH\text{-equations}), \text{Deriv}((\text{ap}_1 \text{thmT reify}(\text{enc}_X \dots)) = \text{reify}(\text{out}_X \dots)).$$

Each thmTDispX proves that thmT at the encoded proof tree reduces to the encoded conclusion, given the IHs supplied for the sub-proof trees.

4 Soundification of thmT

The original body_X definitions (modelled on Guard section 3.3) are *unconditional*: they extract sub-proof images from the input pair (a, bb) via *Lift/Comp/Post* combinators and assemble the conclusion. For a malformed input proof tree, the unconditional bodies can produce arbitrary terms via safe-default fallbacks (axFstLf , axSndLf), risking $\text{thmT}(y_{\text{bad}}) = \text{codeFormula}(\perp)$ for some adversarial y_{bad} , which would refute *Con* in the meta-theory and make *godell* vacuous.

4.1 Verifying-body design

For each premise-consuming dispatcher X , we replace body_X with a verifying variant body_X^v of the form

$$\text{body}_X^v = \text{Post verifier}_X \text{Pair},$$

where verifier_X is a Fun_1 over $t = \text{Pair } a \text{ } bb$:

$$\text{verifier}_X = \text{Comp}_2 \text{IfLf} \langle \text{discriminant} \rangle (\text{Comp}_2 \text{Pair} (\text{KT codeTriv}) \langle \text{assembly} \rangle).$$

The discriminant is whichever sub-proof image (or encoding-level projection) the body would otherwise unsafely use. On the leaf (*O*) branch the body outputs $\text{codeTriv} = \text{falseT}$; on the Pair branch it executes the original assembly. The eval-pass lemma $\text{body}_X^v.\text{eval_pass}$ takes a Pair-shape witness for the IH and discharges the *IfLf* via axIfLfN .

4.2 Coverage

Dispatcher	Proto / VProof file	Variants delivered
mp	BRA/SoundMpProto.agda	closed + lifted
ruleInst	BRA/SoundRuleInstProto.agda	closed + lifted
ruleSym	BRA/SoundRuleSymProto.agda	closed + lifted
cong ₁	BRA/SoundCong1Proto.agda	closed
cong _L	BRA/SoundCongLProto.agda	closed + lifted + dl
cong _R	BRA/SoundCongRProto.agda	closed + lifted + dl
ruleTrans	BRA/SoundRuleTransProto.agda	closed + lifted + dl
ruleInst ₂	BRA/SoundRuleInst2Proto.agda	closed + parametric
ruleIndBT	BRA/SoundRuleIndBTProto.agda	closed + closed-param

For each row the body in `BRA/Thm/ThmT.agda` reads `bodyX = bodyXv` and the dispatchers `thmTDispX`, `thmTDispX_param`, ... route through the corresponding `bodyXv.eval_pass` variants supplied with the appropriate Pair-shape witness.

Theorem 6 (Soundness of `thmT`). *For every closed proof tree y , `thmT(y)` is either `reify ∘ codeFormula(P)` for some P , or the explicit safe default `codeTriv = codeFormula(0 = 0)`. In particular, `thmT(y) ≠ codeFormula(⊥)` for any y unless BRA is actually inconsistent.*

Remark 7 (Lifted variants). *Several Theorem 12 schematic proofs (`Parts/Comp2`, `Parts/Fan`, etc.) require Pair-shape hypotheses inside $P \Rightarrow \cdot$ contexts (singly or doubly lifted). We provide `bodyXv.eval_pass.l` and `bodyXv.eval_pass.dl` for `congL`, `congR`, `ruleTrans`, `mp`, `ruleInst`, `ruleSym`; the closed pass suffices for `cong1` and `ruleIndBT`.*

5 Theorem 12: parametric closure

For each primitive-recursive function f definable in BRA, Theorem 12 internalises the equation `thmT(Df(x)) = th(f(x))` inside BRA itself. The implementation uses 15 modular pairs of “Param” wrappers (`BRA/Thm12/Param/AxX.agda`) and “Parts” glue (`BRA/Thm12/Parts/X.agda`), assembled into the `FromBridges` record in `BRA/Thm12.agda`.

Theorem 8 (Thm 12 result records). *The records `Thm12_F1.Result thmT` and `Thm12_F2.Result sub` in `BRA/Thm12.agda` are inhabited unconditionally by `FromBridges.thm12_F1` and `FromBridges.thm12_F2`.*

6 Theorem 14 and the Compose module

Theorem 14 packages the steps 1–5 of Guard’s section 3.5 into a contract on a single function `constr5 : Fun1` together with a parametric internal implication

$$\text{step}_5(x) : \text{Deriv} \left((\text{thmT}(x) = c_G) \Rightarrow (\text{thmT}(\text{constr}_5 x) = \text{codeFormula}(\perp)) \right)$$

where $c_G = \text{reify}(\text{codeFormula}(G))$ is the code of the Gödel sentence. The concrete `constr5_final` and `step5.l` live in `BRA/Th14Step5.agda`. The Compose module (`BRA/GoedelIII.agda`) then produces

Theorem 9 (`godelIII`). *Deriv Con $\rightarrow$ Deriv $\perp$, where $\text{Con} = \neg(\text{ap}_1 \text{thmT}(\text{var } 0) = \text{reify}(\text{codeFormula}(\perp)))$ universally closed in `var 0`.*

The headline file `BRA/GoedelIIIFull.agda` instantiates `Compose` with `constr5_final` and `step5_l`, plugs in `r12_th = FromBridges.thm12_F1 thmT` and `r12_sub = FromBridges.thm12_F2 sub`, and exports

```
godelIII : Deriv Con -> Deriv bot
godelIII = Final.godelIII
```

unconditionally.

7 Implementation notes

- **Build.** Cold rebuild of `BRA/GoedelIIIFull.agda` takes ~ 32 s. Cached typechecks of the headline file and `BRA/Thm/ThmT.agda` are under 1.2s each.
- **Discipline.** Throughout BRA we use only `--safe --without-K --exact-split`. No postulates, no holes, no `with`-abstraction, no dot patterns. Every `let`-binding uses `camelCase` to avoid Agda's mid-identifier underscore mixfix grammar.
- **Soundification scope.** All 9 inductive premise-consuming dispatchers in `BRA/Thm/ThmT.agda` now have `bodyX = bodyXv`. No further dispatcher reads sub-proof images without an `IFlf` gate.
- **Encoding-level safety.** `ruleIndBT`'s body extracts `codeFormula(P)` directly from the encoding (no IH consumed); its verifying variant gates on the encoding's outer `Snd` being `Pair`-shaped, so a forged leaf payload is mapped to `codeTriv`.

8 Files

The complete BRA codebase is in the `BRA/` subdirectory of the project repository. Headline files:

- `BRA/GoedelIIIFull.agda` — top-level `godelIII : Deriv Con → Deriv ⊥`.
- `BRA/GoedelIII.agda` — `Compose` module taking the Theorem 14 contract and producing `godelIII`.
- `BRA/Thm14Abstract.agda` — abstract Theorem 14 tower.
- `BRA/Thm14Step5.agda` — concrete `constr5_final + step5_l`.
- `BRA/Thm12.agda`, `BRA/Thm12/` — Theorem 12 closure (15 Param + Parts pairs).
- `BRA/Thm/ThmT.agda` — `thmT` definition and all `thmTDispX` dispatchers (the abstract block).
- `BRA/Sound*Proto.agda`, `BRA/Sound*VProof.agda` — verifying bodies and their eval-pass lemmas (Section ??).